EVALUACIÓN FINAL INTEGRADORA 2026

PUESTA AL DÍA: ARQUITECTURA, USO DE TRELLO Y DIVISIÓN DE TAREAS

VELOX — Tienda de Calzado Deportivo de Alto Rendimiento.
Sistema Full-Stack desacoplado: Backend API REST en **Django 6 + DRF** e interfaz en **Next.js (App Router)**.

1. CONTEXTO GENERAL: DOS MATERIAS, UN MISMO SISTEMA

El Examen Final Integrador (EFI) se desarrolla en nuestro equipo de **4 integrantes** conectando dos espacios curriculares mediante una API REST. La nota final de la EFI promedia ambas defensas orales:

BACKEND — Ingeniería de Software

Docente: Prof. Facundo Esteban Lemo • Entrega: Lunes 2 de Noviembre

- **Stack cerrado:** Python 3.12+ con `uv`, Django 6.x, DRF, SimpleJWT, `drf-spectacular`, `django-filter`, CORS y **Docker**.
- **Base de Datos: 12 modelos relacionales** (supera el mínimo de 10).
- **API REST: 8 ViewSets**, paginación global, **2 NestedSerializers** en `list` y **3 acciones** `@action` personalizadas.
- **Vistas y Admin:** Template HTML de bienvenida en `/`, panel `/admin/` completo y Swagger en `/api/docs/`.
- **Defensa Oral (Martes 3/11):** Todos los integrantes defienden el proceso paso a paso y el código de Django.

FRONTEND — Programación III

Docente: Prof. Valentino Arballo • Entrega: Jueves 5 de Noviembre

- **Stack cerrado:** Next.js (App Router), React 19, **Tailwind CSS v4 puro** (prohibido MUI/Shadcn/Redux), React Hook Form, Zod y Vitest.
- **Enrutamiento (R1):** 6 rutas principales, 1 anidada, 2 dinámicas (`[id]`), layout raíz + layout anidado, `loading` / `error` / `not-found`.
- **Componentes (R2) y CRUD (R3):** 8 componentes reutilizables (Server vs Client justificados) y las **4 operaciones CRUD** (`GET, POST, PUT/PATCH, DELETE`).
- **Auth (R4), Forms (R5), Context (R6), Tests (R7) y Deploy (R8):** JWT en Cookies HTTP-Only, `CartContext`, 4 tests en Vitest y deploy en **Vercel**.

2. IDENTIDAD DE MARCA: VELOX (DARK MODE NATIVO)

Concepto Core: Velocidad sin fricción y precisión técnica. El *Modo Oscuro* es el estado natural de la marca. Tono de voz enérgico en botones: "*Acelerar*", "*Analizar rendimiento*", "*Adquirir*".

Tipografías: `Space Grotesk` (Títulos y números) + `Inter` (Cuerpo e inputs). Fotografía de calzado en PNG transparente con glow verde neón.

Base	Surface	Accent	Text 1	Text 2
#121212	#2A2A2A	#39FF14	#FFFFFF	#D1D5DB

3. CRONOGRAMA CRÍTICO DE ENTREGAS (OCTUBRE – NOVIEMBRE 2026)

FECHA LÍMITE	HITO / SPRINT	OBJETIVO PRINCIPAL DEL EQUIPO
Miércoles 28/10	Etapa 0 (Completada)	Definición de dominio VELOX, Diagrama ER (12 modelos), reglas de ingeniería y tablero Trello.
Viernes 30/10	Cierre Sprint 1	Setup Django + Docker + `uv`, App Usuarios (JWT), 8 Modelos de Catálogo y Setup Next.js + UI Base.
Domingo 01/11	Cierre Core API	Los 12 modelos, 8 ViewSets, 2 NestedSerializers, 3 `@action`, filtros y App Pedidos/Reseñas listos.
LUNES 02/11	ENTREGA BACKEND	**Entrega final Ing. de Software:** Swagger `/api/docs/`, deploy público del Back y README con capturas.
Martes 03/11	Defensa Back + Sprint Front	Inicio de defensas con Prof. Lemo. Todo el equipo pasa a integrar rutas de Next.js con la API.
JUEVES 05/11	ENTREGA FRONTEND	**Entrega final Programación III:** CRUD 4 ops, Auth HTTP-Only, CartContext, 4 tests Vitest y Vercel.

4. CÓMO USAMOS NUESTRO TABLERO TRELLO (KANBAN)

Tanto Ingeniería de Software como Programación III evalúan la organización del equipo mediante nuestro tablero Kanban: `https://trello.com/b/frG8yKLI/efi-django-next`. El tablero está estructurado en **6 columnas (listas)** que toda tarjeta debe recorrer de izquierda a derecha:

1. 📋 Backlog
Todas las tareas del proyecto priorizadas con su checklist técnico.

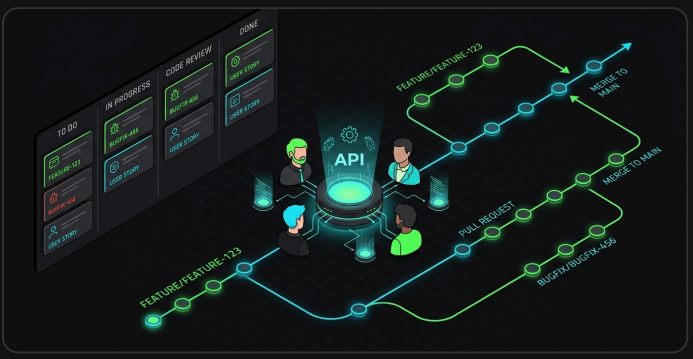
2. 📌 Sprint Actual
Las tareas asignadas para completar en los próximos 2 días.

3. 🔨 En Desarrollo
Cuando empezás a codear tu rama, movés tu tarjeta acá y te auto-asignás.

4. 👀 Code Review
Abriste el Pull Request (PR) hacia `develop` y pegaste el link en la card.

5. 🚫 Bloqueado
Si dependés de otro endpoint o tenés un bug crítico, dejá comentario.

6. ✅ Done
PR revisado por un compañero, bitácora actualizada y mergeado a `develop`.



🏷️ Código de Etiquetas (Labels) en Trello

Cada tarjeta tiene etiquetas para identificar de un vistazo qué capa toca:

Backend (Django) | Frontend (Next.js) | API / Integración | Base de Datos | UI / Marca VELOX | Testing (Vitest) | DevOps / Deploy | Documentación | Urgente / Bug

Regla de asignación: Nadie empieza a programar una funcionalidad sin que su tarjeta esté en 🔨 **En Desarrollo** con su usuario asignado.

5. FLUJO GITFLOW UNIFICADO (PROHIBIDO PUSHEAR DIRECTO A MAIN/DEVELOP)

🌿 Estructura de Ramas

- `main` : Versión estable de entrega final y producción.
- `develop` : Rama de integración continua del equipo.
- `feature/<area>-<descripcion>` : Nace desde `develop` por cada tarjeta de Trello (ej. `feature/back-usuarios-auth`).
- `fix/<area>-<descripcion>` : Corrección de errores de integración.

📝 Formato de Commits (Conventional Commits)

- `feat(productos): agregar modelo VarianteProducto con SKU`
- `feat(cart): implementar CartContext con localStorage`
- `fix(pedidos): restituir stock en @action cancelar_pedido`
- `docs(bitacora): registrar decisiones técnicas del PR #4`
- `test(zod): agregar 4 tests unitarios con Vitest`

6. LAS 3 REGLAS DE ORO INNEGOCIABLES (DEFENSA ORAL Y CONTRATO API)

1. Documentación Viva en cada PR

Ningún PR se aprueba sin actualizar **en el mismo PR** el `README.md` (endpoints/rutas y capturas) y `docs/bitacora_defensa.md` explicando qué se hizo y cómo defenderlo ante los profes Lemo y Arballo.

2. Comentarios de Defensa en Código

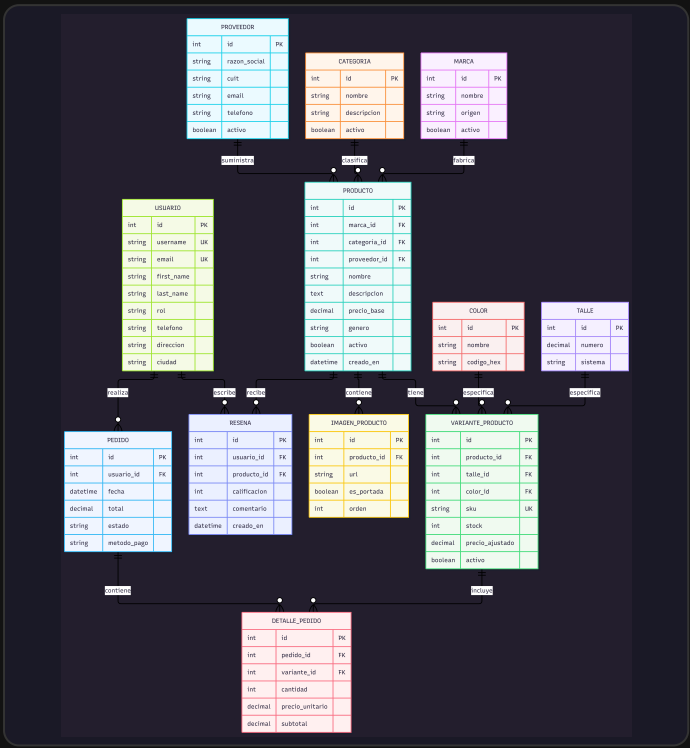
En Next.js: 1 línea arriba de cada componente indicando si es `Server` o `Client ("use client")` y por qué.
En Django: Docstring breve en cada `ViewSet` , `Serializer` y permiso explicando su lógica.

3. Contrato API Estricto

Paridad `snake_case` **:** RHF y Zod usan los mismos nombres de campos que DRF (`precio_base` , `metodo_pago`).
Paginación DRF: Todo listado devuelve `{ count, next, previous, results }` (iterar siempre `.results`).

7. MODELO DE BASE DE DATOS RELACIONAL (12 MODELOS EN 3FN)

La consigna de Ingeniería de Software exige un mínimo de 10 modelos. Nuestro diseño implementa **12 modelos divididos en 3 apps de Django** (`usuarios` , `productos` y `pedidos`), separando el calzado conceptual (`PRODUCTO`) de la unidad física en stock (`VARIANTE_PRODUCTO` con **Talle + Color + SKU único**):



🧠 Argumentos Clave para Defender el DER

- **1. Custom User Model (`USUARIO`):** Hereda de `AbstractUser` y suma el campo `rol` (`ADMIN` , `EMPLEADO` , `CLIENTE`) desde la migración cero (`AUTH_USER_MODEL`). Evita joins innecesarios al validar permisos en DRF.
- **2. Desacople `PRODUCTO` vs `VARIANTE_PRODUCTO` :** El stock real no pertenece al nombre "Velox Carbon", sino a la combinación exacta de **Producto + Talle + Color** identificada por un `sku` único.
- **3. Galería `IMAGEN_PRODUCTO` (1:N):** Cumple la 1ra Forma Normal (1FN) permitiendo múltiples fotos ordenadas y marcando la principal con `es_portada` .
- **4. Trazabilidad en `DETALLE_PEDIDO` :** Apunta a `variante_id` y congela `precio_unitario` al momento de la compra.

🔌 Cobertura de Requisitos API REST (DRF)

- **8 ViewSets expuestos:** `Producto` , `Marca` , `Categoria` , `VarianteProducto` , `Pedido` , `Usuario` , `Proveedor` y `Resena` .
- **2 NestedSerializers en `list` :** Sobrescribiendo `get_serializer_class()` en `ProductoViewSet` y `PedidoViewSet` .
- **3 Acciones `@action` :** `destacados` (GET), `stock_por_talle` (GET) y `cancelar_pedido` (POST que restituye stock).
- **Filtros + Paginación + Auth Dual:** `django-filter` en catálogo, `PageNumberPagination` , SessionAuth + SimpleJWT.

8. ARQUITECTURA DEL FRONTEND EN NEXT.JS (REQUISITOS R1 A R8)

REQUISITO PROGRAMACIÓN III	IMPLEMENTACIÓN EN VELOX	ARCHIVOS CLAVE EN `FRONTEND/SRC/`
R1. Enrutamiento y Layouts	6 rutas principales (`/` , `/productos` , `/tecnologia` , `/login` , `/carrito` , `/dashboard`), 1 anidada (`/dashboard/pedidos`), 2 dinámicas (`/productos/[id]` , `/dashboard/pedidos/[id]`) + layout anidado.	`app/layout.jsx` , `app/dashboard/layout.jsx` , `loading.jsx` , `error.jsx` , `not-found.jsx`
R2. 8 Componentes (Server vs Client)	Server: páginas de catálogo, `Footer` , `Badge` , `SkeletonCard` . Client (`"use client"`): `Navbar` , `FilterBar` , `Modal` , `Button` , `ProductCard` .	`components/Navbar.jsx` , `ProductCard.jsx` , `FilterBar.jsx` , `Modal.jsx` , etc.
R3. API y 4 Operaciones CRUD	Lectura paginada con filtros + **Crear (POST), Leer (GET), Editar (PUT/PATCH) y Borrar (DELETE)** sobre un recurso principal en `/dashboard` o reseñas, más creación y cancelación de pedidos.	`lib/api.js` , `app/actions.js` (Server Actions con `revalidatePath`)
R4 + R5. Auth JWT y Formularios	Login con Server Action guardando JWT en **Cookie HTTP-Only**. Mínimo 2 formularios controlados con **React Hook Form + Zod** en `snake_case` .	`lib/auth.js` , `validations/authSchema.js` , `validations/orderSchema.js`
R6 + R7 + R8. Context, Tests y Deploy	`CartContext` sincronizado con `localStorage` . Mínimo **4 tests unitarios con Vitest**. Deploy en **Vercel** conectado al Backend público.	`context/CartContext.jsx` , `app/__tests__/*.test.js` , `vitest.config.mts`

9. DIVISIÓN DEL EQUIPO: 📌 SPRINT ACTUAL (CIERRE: VIERNES 30/10)

Como el **Backend se entrega primero (Lunes 2 de Noviembre)** y el Frontend necesita consumir la API real antes del Jueves 5 de Noviembre, dividimos el **Sprint 1** en **4 tarjetas paralelas sin conflictos de archivos** (3 de Backend + 1 de Frontend):

👤 INTEGRANTE 1 — Base Backend & Docker `Backend`

Tarjeta Trello: 🚀 **Setup Django 6 + uv + Docker + Estructura nucleo/apps + Vista bienvenida (/)**

Rama: `feature/back-setup-entorno`

- Inicializar entorno con `uv` (`pyproject.toml` y `uv.lock`).
- Crear `Dockerfile` y `docker-compose.yml` operativos.
- Estructurar `src/nucleo/` (settings, CORS, paginación global) y `src/apps/`.
- Crear la vista de bienvenida con template HTML en la raíz (`/`).

👤 INTEGRANTE 2 — Usuarios, Roles & JWT `Backend`

Tarjeta Trello: 🔐 **App Usuarios: Custom User (AbstractUser + rol) + Auth Dual + UsuarioViewSet**

Rama: `feature/back-usuarios-auth`

- Implementar `Usuario(AbstractUser)` con campo `rol` (`ADMIN` , `EMPLEADO` , `CLIENTE`) en `src/apps/usuarios/` .
- Configurar `SessionAuthentication` + `SimpleJWT` (`/api/token/`).
- Crear permisos `BasePermission` por rol y exponer `UsuarioViewSet` (1/8).

👤 INTEGRANTE 3 — Modelos de Catálogo & Admin `DB + Back`

Tarjeta Trello: 👟 **App Productos: 8 Modelos (Marca, Categoria, Proveedor, Producto, Talle, Color, Variante SKU, Imagen) + Admin**

Rama: `feature/back-modelos-catalogo`

- Codificar los 8 modelos de catálogo en `src/apps/productos/models.py` con SKU único.
- Configurar `admin.py` con filtros, búsqueda y ordenamiento.
- Crear fixture/seed con zapatillas VELOX de prueba.

👤 INTEGRANTE 4 — Setup Next.js & UI VELOX `Frontend`

Tarjeta Trello: 🎨 **Setup Next.js App Router + Tokens Dark Mode VELOX + Layouts (R1) + Componentes UI Base (R2)**

Rama: `feature/front-setup-identidad`

- Inicializar Next.js + Tailwind v4 con tokens `#121212` , `#2A2A2A` y `#39FF14` .
- Crear `layout.jsx` raíz, `dashboard/layout.jsx` anidado, `loading` , `error` y `not-found` (R1).
- Construir `Navbar` , `Footer` , `Button` , `Badge` y `SkeletonCard` (R2).

10. HOJA DE RUTA DEL 📋 BACKLOG (SPRINTS 2 Y 3 HASTA LAS ENTREGAS)

#	TARJETA EN EL BACKLOG DE TRELLO	RAMA GITFLOW	ÁREA	VENCIMIENTO
B1	📦 ViewSets de Catálogo (5 ViewSets) + NestedSerializer en ProductoViewSet + Filtros + 2 `@action` (`destacados` , `stock_por_talle`)	`feature/back-api-productos`	Backend	Dom 01/11
B2	🛒 App Pedidos y Reseñas: 3 Modelos (Pedido, DetallePedido, Resena) + NestedSerializer + `@action cancelar_pedido` (restituye stock)	`feature/back-pedidos-resenas`	Backend	Dom 01/11
B3	📄 [ENTREGA 2/11] Documentación OpenAPI/Swagger (`/api/docs/`) + Auditoría EFI + Deploy Público + README con Capturas	`feature/back-docs-deploy`	Deploy Back	Lun 02/11
F1	🗂️ Catálogo y Rutas (R1-R3): Home (`/`), Catálogo (`/productos`), Ficha Dinámica (`/productos/[id]`), Tecnología + `FilterBar` y `ProductCard`	`feature/front-catalogo-rutas`	Frontend	Mar 03/11
F2	🔑 Autenticación JWT (R4) + Cookies HTTP-Only en Server Actions + Formulario `/login` con React Hook Form y Zod (R5)	`feature/front-auth-jwt`	Frontend	Mar 03/11
F3	🛒 Estado Global `CartContext` (R6) + Página `/carrito` y Checkout con React Hook Form + `OrderSchema` Zod (R5)	`feature/front-carrito-context`	Frontend	Mié 04/11
F4	⚙️ Dashboard (`/dashboard` y `/dashboard/pedidos/[id]`) + 4 Operaciones CRUD Completas (`GET, POST, PUT/PATCH, DELETE`) (R1, R3)	`feature/front-dashboard-crud`	Frontend	Mié 04/11
F5	🧪 [ENTREGA 5/11] Testing Unitario con Vitest (4 tests R7) + Despliegue en Vercel (R8) + README Final y Capturas	`feature/front-tests-deploy`	Tests + Deploy	Jue 05/11

🚀 Próximo Paso Inmediato para Cada Integrante

1) Entrar al tablero Trello (`https://trello.com/b/frG8yKLI/efi-django-next`). **2)** Elegir una de las 4 tarjetas de 📌 **Sprint Actual**, asignarse como miembro y moverla a 🔨 **En Desarrollo**. **3)** Crear su rama `feature/...` desde `develop` y recordar actualizar el `README.md` y `docs/bitacora_defensa.md` antes de abrir su Pull Request.